

Boolean functions:

applications in quantum error correction and beyond

Bohan Lu Duke University

Correcting one bit of error in memory



A memory controller writes an encoded block and checks its constraints when the block is read. A physical disturbance may change a stored 0 to 1, or 1 to 0.

The *encoder* maps four message bits to eight stored bits. The *decoder* maps the eight values read from memory back to four message bits.

Assumption: at most one stored bit changes, at a position unknown to the decoder.

We will construct all 16 allowed eight-bit words and a decoder that corrects any one changed bit.

The encoder evaluates eight XOR expressions

For bits, $\oplus$ denotes XOR: $0 \oplus 0 = 1 \oplus 1 = 0$ and $0 \oplus 1 = 1 \oplus 0 = 1$. The message is (b, a_1, a_2, a_3) . A stored position is an address $u = (u_1, u_2, u_3)$. The product $a_i u_i$ equals u_i when $a_i = 1$ and equals 0 when $a_i = 0$.

$$f_{a,b}(u) = b \oplus a_1 u_1 \oplus a_2 u_2 \oplus a_3 u_3.$$

u	000	001	010	011
$f_{a,b}(u)$	b	$b \oplus a_3$	$b \oplus a_2$	$b \oplus a_2 \oplus a_3$
$(b, a_1, a_2, a_3) = (0, 1, 1, 1)$	0	1	1	0

u	100	101	110	111
$f_{a,b}(u)$	$b \oplus a_1$	$b \oplus a_1 \oplus a_3$	$b \oplus a_1 \oplus a_2$	$b \oplus a_1 \oplus a_2 \oplus a_3$
$(b, a_1, a_2, a_3) = (0, 1, 1, 1)$	1	0	0	1

Thus the example message is stored as 01101001 in address order 000, 001, ..., 111.

The four message bits still contain all the information. The eight stored values add constraints that make corruption detectable and correctable.

Affine Boolean functions define the codewords

A *Boolean function* maps each binary address to one bit. A constant XOR a binary linear combination of the inputs is an *affine Boolean function*. Thus every message defines the affine function $f_{a,b}$ on the preceding slide.

The *encoder* is the evaluation map

$$\text{Enc}(b, a_1, a_2, a_3) = (f_{a,b}(000), f_{a,b}(001), \dots, f_{a,b}(111)).$$

The message can be recovered from an uncorrupted evaluation word:

$$b = f_{a,b}(000), \quad a_i = f_{a,b}(e_i) \oplus f_{a,b}(000),$$

where e_i has a 1 in position i and zeros elsewhere.

Therefore different four-bit messages give different eight-bit words. These 16 words are the *codewords*, and their set is the code C .

C contains 16 selected words among all $2^8 = 256$ possible eight-bit words.

From a truth table to a polynomial

The *algebraic normal form* (ANF) is the unique XOR of products of distinct input variables representing a Boolean function. A product of bits is their AND. The *degree* is the largest number of variables in one nonzero product.

Let $p(u) = u_1 \oplus u_2 \oplus u_3$ be the encoded example. Let r be the Boolean function whose table is the eight bits read from memory:

$$p \longleftrightarrow 01101001, \quad r \longleftrightarrow 11101001.$$

Only the value at address 000 changed. Define

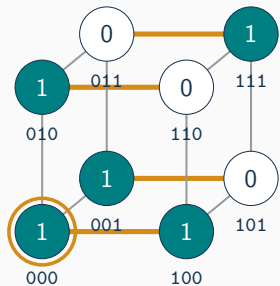
$$\delta_{000}(u) = (1 \oplus u_1)(1 \oplus u_2)(1 \oplus u_3), \quad \delta_{000}(000) = 1, \quad \delta_{000}(u) = 0 \ (u \neq 000).$$

Then

$$r = p \oplus \delta_{000} = 1 \oplus u_1 u_2 \oplus u_1 u_3 \oplus u_2 u_3 \oplus u_1 u_2 u_3.$$

Every codeword has degree at most one. The readout has degree three, so $r \notin C$.

Comparing neighboring table entries



Below: address. Teal: 1. White: 0.
Gold: first-bit edges. Ring: changed entry.

The direction $e_1 = (1, 0, 0)$ toggles the first address bit. The *Boolean derivative* compares the two endpoint values:

$$D_{e_i} f(u) = f(u \oplus e_i) \oplus f(u).$$

At the edge containing the changed address,

$$p(000) \oplus p(100) = 0 \oplus 1 = 1, \quad r(000) \oplus r(100) = 1 \oplus 1 = 0.$$

Derivatives of p and r along e_1 .

edge	000, 100	001, 101	010, 110	011, 111
$D_{e_1} p$	1	1	1	1
$D_{e_1} r$	0	1	1	1

Only the comparison using address 000 changes. The other three pairs have the same endpoints in p and r . The same statement holds in each coordinate direction.

Recovering the four message coefficients

Toggling u_i cancels every affine term except $a_i u_i$:

$$D_{e_i} f_{a,b} = a_i.$$

The four disjoint e_1 edges give four estimates of a_1 :

addresses	000, 100	001, 101	010, 110	011, 111
received XOR	0	1	1	1

Question: Why is this majority reliable?

Recovering the four message coefficients

Toggling u_i cancels every affine term except $a_i u_i$:

$$D_{e_i} f_{a,b} = a_i.$$

The four disjoint e_1 edges give four estimates of a_1 :

addresses	000,100	001,101	010,110	011,111
received XOR	0	1	1	1

One changed address lies on exactly one e_i edge. Hence at least three of the four votes equal a_i .

Thus majority gives $a_1 = a_2 = a_3 = 1$.

$$r(u) \oplus a_1 u_1 \oplus a_2 u_2 \oplus a_3 u_3 \quad \longleftrightarrow \quad 10000000.$$

Seven entries now equal the constant $b = 0$. The decoded message is $(0, 1, 1, 1)$.

Three of four votes recover each a_i . Seven residual entries recover b .

Support intersections compute Hamming distance

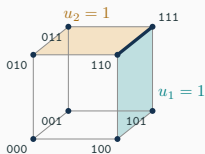
For a Boolean function f on the eight addresses, define its support and weight. For two evaluation words, define their *Hamming distance*:

$$\text{supp}(f) = \{u : f(u) = 1\},$$

$$\text{wt}(f) = |\text{supp}(f)| = \sum_{u \in \mathbb{F}_2^3} f(u),$$

$$d_H(f, g) = |\{u : f(u) \neq g(u)\}|,$$

$$d_H(f, g) = \text{wt}(f \oplus g).$$



The coordinate functions u_1 and u_2 each have four ones.
Their product selects the two addresses on the common edge:

$$\text{wt}(u_1) = \text{wt}(u_2) = 4, \quad \text{wt}(u_1 u_2) = 2.$$

$$\text{wt}(f \oplus g) = \text{wt}(f) + \text{wt}(g) - 2 \text{wt}(fg), \quad \underbrace{d_H(u_1, u_2) = 4 + 4 - 2(2) = 4}_{\text{one particular pair}}.$$

The code's minimum distance is four

The *minimum distance of the code* is

$$d_{\min}(C) = \min_{\substack{f, g \in C \\ f \neq g}} d_H(f, g).$$

For $f \neq g$, $h = f \oplus g$ is nonzero and affine.

If h has coefficient $a_i = 1$, each pair $\{u, u \oplus e_i\}$ contains one output 1. Four pairs give $\text{wt}(h) = 4$. The remaining case $h = 1$ has weight 8.

Thus all distinct pairs have distance 4 or 8. Slide 7 attains 4, so $d_{\min}(C) = 4$.

For parity p , let

$$H_c = \{u : p(u) = c\}.$$

Such a level set of a nonzero linear Boolean function is a hyperplane. H_0 contains zero.

The set $H_1 = \{001, 010, 100, 111\}$ is its translate, an *affine hyperplane*.

The code's minimum distance is four

The *minimum distance of the code* is

$$d_{\min}(C) = \min_{\substack{f, g \in C \\ f \neq g}} d_H(f, g).$$

For $f \neq g$, $h = f \oplus g$ is nonzero and affine.

If h has coefficient $a_i = 1$, each pair $\{u, u \oplus e_i\}$ contains one output 1. Four pairs give $\text{wt}(h) = 4$. The remaining case $h = 1$ has weight 8.

Thus all distinct pairs have distance 4 or 8. Slide 7 attains 4, so $d_{\min}(C) = 4$.

If f and g were each within one change of the same readout r , then

$$4 = d_{\min}(C) \leq d_H(f, g) \leq d_H(f, r) + d_H(r, g) \leq 2,$$

a contradiction. One-error correction is unique.

For parity p , let

$$H_c = \{u : p(u) = c\}.$$

Such a level set of a nonzero linear Boolean function is a hyperplane. H_0 contains zero.

The set $H_1 = \{001, 010, 100, 111\}$ is its translate, an *affine hyperplane*.

What the classical example established

object	calculation	conclusion
encoder	$(b, a) \mapsto (f_{a,b}(u))_u$	four bits determine eight constrained values
readout	$r = p \oplus \delta_{000}$	ANF records the changed entry
derivative	$D_{e_i} f_{a,b} = a_i$	four local votes recover each a_i
code	$d_{\min}(C) = 4$	every one-bit error has a unique correction

For the displayed readout,

$$\underbrace{d_H(p, r) = 1}_{\text{one particular codeword and readout}}, \quad \underbrace{d_{\min}(C) = 4}_{\text{smallest separation of two codewords}}.$$

These two distances answer different questions. The first counts the observed corruption. The second is a property of the entire code.

Single-qubit operations change bits and phases

A qubit is a unit vector in $\mathbb{C}^2$, with computational basis $|0\rangle = (1, 0)^T$ and $|1\rangle = (0, 1)^T$. A bit flip X exchanges these basis vectors. A phase flip Z changes their relative sign:

$$X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle, \quad \frac{|0\rangle + |1\rangle}{\sqrt{2}} \xrightarrow{Z} \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$

Let $\omega = e^{i\pi/4}$. The rotation $T = \text{diag}(1, \omega)$ multiplies the $|1\rangle$ component by ω , while $T^\dagger$ uses ω^{-1} . Applying T independently to all n qubits is written $T^{\otimes n}$. For an n -bit physical word v ,

$$T^{\otimes n}|v\rangle = \omega^{\text{wt}(v)}|v\rangle.$$

On three logical qubits, the controlled-controlled- Z gate changes only the sign of label 111:

$$\text{CCZ}|a_1 a_2 a_3\rangle = (-1)^{a_1 a_2 a_3} |a_1 a_2 a_3\rangle.$$

Encoding three qubits with complementary words

For each three-bit label a , pair the affine words with constants $b = 0$ and $b = 1$:

label a	$f_{a,0}$	$f_{a,1}$
000	00000000	11111111
100	00001111	11110000
111	01101001	10010110

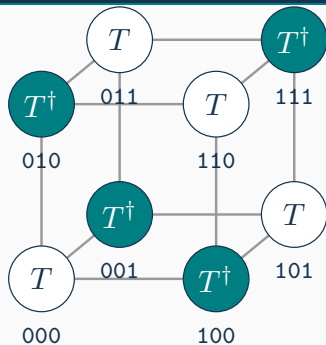
The encoded basis state is

$$|a\rangle_L = \frac{|f_{a,0}\rangle + |f_{a,1}\rangle}{\sqrt{2}}.$$

The eight disjoint pairs give eight orthogonal states, encoding three logical qubits in eight physical qubits. Because $f_{a,1} = f_{a,0} \oplus \mathbf{1}$, flipping all eight physical bits exchanges the two terms and leaves $|a\rangle_L$ unchanged.

A physical phase operation preserves $|a\rangle_L$ only if both words in its pair receive the same phase.

Choosing T and $T^\dagger$ with parity



Below: physical address u . White:
 $p(u) = 0$, apply T . Teal: $p(u) = 1$,
 apply $T^\dagger$.

The word $v = p$ has its four ones at $T^\dagger$ sites, giving total angle $-4(\pi/4)$.

For any physical word v , count its ones at T sites positively and its ones at $T^\dagger$ sites negatively. Its *signed weight* is

$$\text{sw}_p(v) = \sum_{u \in \mathbb{F}_2^3} (-1)^{p(u)} v(u).$$

The sum is an ordinary integer. The physical phase is $\omega^{\text{sw}_p(v)}$.

The rotation depends only on the address. Both words of every encoded pair must be checked.

Checking the eight-qubit logical gate

At one address, $p = 0$ contributes v , while $p = 1$ contributes $-v$. In both cases,

$$(-1)^{p(u)}v(u) = (p \oplus v)(u) - p(u).$$

Summing over addresses gives $\text{sw}_p(v) = \text{wt}(p \oplus v) - 4$. For $a \neq 111$, $p \oplus f_{a,b}$ is nonconstant affine and has weight four.

logical label a	$\text{sw}_p(f_{a,0})$	$\text{sw}_p(f_{a,1})$	phase
$a \neq 111$	0	0	
$a = 111$	-4	+4	

Question: Do +4 and -4 give different phases when each count contributes an angle of $\pi/4$?

Checking the eight-qubit logical gate

At one address, $p = 0$ contributes v , while $p = 1$ contributes $-v$. In both cases,

$$(-1)^{p(u)}v(u) = (p \oplus v)(u) - p(u).$$

Summing over addresses gives $\text{sw}_p(v) = \text{wt}(p \oplus v) - 4$. For $a \neq 111$, $p \oplus f_{a,b}$ is nonconstant affine and has weight four.

logical label a	$\text{sw}_p(f_{a,0})$	$\text{sw}_p(f_{a,1})$	phase
$a \neq 111$	0	0	+1
$a = 111$	-4	+4	-1

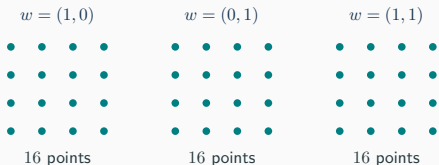
$\omega^4 = \omega^{-4} = -1$. Both words in every pair receive the same phase, and exactly the logical label 111 changes sign. The physical rotations implement logical CCZ.

Deleting a subspace leaves weights 24 and 32

The eight-qubit encoding does not correct every one-qubit error. The protected construction splits each six-bit address into $u \in \mathbb{F}_2^4$ and $w \in \mathbb{F}_2^2$.

Delete the subspace $w = 00$, retaining

$$\mathcal{P} = \{(u, w) : w \neq 0\}.$$



$$64 \text{ points} - 16 \text{ deleted} = 48.$$

Each dot is one address. Each block contains all 16 choices of u .

For $\alpha \in \mathbb{F}_2^4$ and $\beta \in \mathbb{F}_2^2$, let $\alpha \cdot u$ and $\beta \cdot w$ denote binary dot products. Define

$$\ell(u, w) = \alpha \cdot u \oplus \beta \cdot w.$$

Every nonzero ℓ has 32 ones on all 64 addresses. On the deleted subspace, $\ell = \alpha \cdot u$, with eight ones if $\alpha \neq 0$ and zero otherwise. The restriction $\ell|_{\mathcal{P}}$ means evaluation only on retained addresses. Therefore

$$\text{wt}(\ell|_{\mathcal{P}}) = \begin{cases} 32 - 8 = 24, & \alpha \neq 0, \\ 32 - 0 = 32, & \alpha = 0, \beta \neq 0. \end{cases}$$

Completing the 48-qubit code

The six coordinate functions give evaluation words $S_1, \dots, S_6$. Let S be their 64 possible XOR combinations. Flipping the sites selected by S_i sends $|v\rangle$ to $|v \oplus S_i\rangle$ and permutes the 64 terms below. An operation that leaves every encoded state unchanged is a *stabilizer check*.

Three logical functions give words K_1, K_2, K_3 . For $a \in \mathbb{F}_2^3$, let $K(a) = a_1 K_1 \oplus a_2 K_2 \oplus a_3 K_3$:

$$|a\rangle_L = \frac{1}{8} \sum_{s \in S} |K(a) \oplus s\rangle.$$

The sign function γ selects T at value 0 and $T^\dagger$ at value 1.

$w = (1,0)$	$w = (0,1)$	$w = (1,1)$																																																
<table> <tr><td>T</td><td>T</td><td>T</td><td>T</td></tr> <tr><td>$T^\dagger$</td><td>T</td><td>$T^\dagger$</td><td>T</td></tr> <tr><td>T</td><td>$T^\dagger$</td><td>$T^\dagger$</td><td>T</td></tr> <tr><td>$T^\dagger$</td><td>$T^\dagger$</td><td>T</td><td>T</td></tr> </table>	T	T	T	T	$T^\dagger$	T	$T^\dagger$	T	T	$T^\dagger$	$T^\dagger$	T	$T^\dagger$	$T^\dagger$	T	T	<table> <tr><td>T</td><td>$T^\dagger$</td><td>T</td><td>T</td></tr> <tr><td>T</td><td>$T^\dagger$</td><td>T</td><td>T</td></tr> <tr><td>$T^\dagger$</td><td>$T^\dagger$</td><td>T</td><td>$T^\dagger$</td></tr> <tr><td>T</td><td>T</td><td>$T^\dagger$</td><td>T</td></tr> </table>	T	$T^\dagger$	T	T	T	$T^\dagger$	T	T	$T^\dagger$	$T^\dagger$	T	$T^\dagger$	T	T	$T^\dagger$	T	<table> <tr><td>T</td><td>$T^\dagger$</td><td>$T^\dagger$</td><td>$T^\dagger$</td></tr> <tr><td>$T^\dagger$</td><td>$T^\dagger$</td><td>T</td><td>$T^\dagger$</td></tr> <tr><td>$T^\dagger$</td><td>T</td><td>T</td><td>T</td></tr> <tr><td>$T^\dagger$</td><td>$T^\dagger$</td><td>T</td><td>$T^\dagger$</td></tr> </table>	T	$T^\dagger$	$T^\dagger$	$T^\dagger$	$T^\dagger$	$T^\dagger$	T	$T^\dagger$	$T^\dagger$	T	T	T	$T^\dagger$	$T^\dagger$	T	$T^\dagger$
T	T	T	T																																															
$T^\dagger$	T	$T^\dagger$	T																																															
T	$T^\dagger$	$T^\dagger$	T																																															
$T^\dagger$	$T^\dagger$	T	T																																															
T	$T^\dagger$	T	T																																															
T	$T^\dagger$	T	T																																															
$T^\dagger$	$T^\dagger$	T	$T^\dagger$																																															
T	T	$T^\dagger$	T																																															
T	$T^\dagger$	$T^\dagger$	$T^\dagger$																																															
$T^\dagger$	$T^\dagger$	T	$T^\dagger$																																															
$T^\dagger$	T	T	T																																															
$T^\dagger$	$T^\dagger$	T	$T^\dagger$																																															
$6\ T^\dagger$	$6\ T^\dagger$	$10\ T^\dagger$																																																

Actual signs: gold T , teal $T^\dagger$. Rows $u_1 u_2$, columns $u_3 u_4$, ordered 00, 01, 11, 10.

The physical sites, logical functions, and rotation signs are separate parts of the construction.

Checking phases through row intersections

A *generator row* is one of the nine evaluation words $K_1, K_2, K_3, S_1, \dots, S_6$. For distinct rows g, h, q , products mean coordinatewise AND and select intersections of their supports. At each physical site, binary XOR has the integer expansion

$$g \oplus h \oplus q = g + h + q - 2(gh + gq + hq) + 4ghq.$$

For a word v , with values v_j and γ_j at site j , define $\text{sw}_\gamma(v) = \sum_{j=1}^{48} (-1)^{\gamma_j} v_j$. Congruence modulo m means equality of integer remainders after division by m .

$$\text{sw}_\gamma(g) \equiv 0 \pmod{8} \qquad \text{coefficient 1,}$$

Checking phases through row intersections

A *generator row* is one of the nine evaluation words $K_1, K_2, K_3, S_1, \dots, S_6$. For distinct rows g, h, q , products mean coordinatewise AND and select intersections of their supports. At each physical site, binary XOR has the integer expansion

$$g \oplus h \oplus q = g + h + q - 2(gh + gq + hq) + 4ghq.$$

For a word v , with values v_j and γ_j at site j , define $\text{sw}_\gamma(v) = \sum_{j=1}^{48} (-1)^{\gamma_j} v_j$. Congruence modulo m means equality of integer remainders after division by m .

$$\begin{aligned} \text{sw}_\gamma(g) &\equiv 0 \pmod{8} && \text{coefficient } 1, \\ \text{sw}_\gamma(gh) &\equiv 0 \pmod{4} && \text{coefficient } -2, \end{aligned}$$

Checking phases through row intersections

A *generator row* is one of the nine evaluation words $K_1, K_2, K_3, S_1, \dots, S_6$. For distinct rows g, h, q , products mean coordinatewise AND and select intersections of their supports. At each physical site, binary XOR has the integer expansion

$$g \oplus h \oplus q = g + h + q - 2(gh + gq + hq) + 4ghq.$$

For a word v , with values v_j and γ_j at site j , define $\text{sw}_\gamma(v) = \sum_{j=1}^{48} (-1)^{\gamma_j} v_j$. Congruence modulo m means equality of integer remainders after division by m .

$$\text{sw}_\gamma(g) \equiv 0 \pmod{8} \quad \text{coefficient } 1,$$

$$\text{sw}_\gamma(gh) \equiv 0 \pmod{4} \quad \text{coefficient } -2,$$

$$\text{sw}_\gamma(ghq) \equiv \begin{cases} 1 & \pmod{2}, \quad \{g, h, q\} = \{K_1, K_2, K_3\}, \\ 0 & \pmod{2}, \quad \text{otherwise.} \end{cases}$$

Terms involving four or more rows have coefficients divisible by eight. Only the logical triple contributes $4a_1a_2a_3$ modulo eight, producing CCZ.

The resulting quantum code

The construction uses 48 physical qubits and encodes 3 logical qubits.

The *quantum distance* d_Q is the smallest number of physical qubits on which an error undetected by every stabilizer check can act while changing the encoded information. Here $d_Q = 3$, so the code corrects an arbitrary error on one physical qubit.

The standard shorthand for these facts is $[[48, 3, 3]]$.

The displayed pattern of 26 rotations T and 22 rotations $T^\dagger$ gives the same phase to all 64 physical words in each encoded basis state and implements logical CCZ.

The explicit Boolean functions define the code. The signed row-intersection conditions verify its logical gate.

Current paper: $\text{rank } S = 6$, $\text{rank } G = 9$, $d_X = 16$, $d_Z = 3$, fixed logical functions, and explicit sign pattern.

Reed–Muller codes and finite-length bounds

Our classical code evaluates affine functions on three-bit addresses. It is the *Reed–Muller code* $\text{RM}(1, 3)$. More generally, $\text{RM}(d, m)$ evaluates degree-at-most- d Boolean polynomials on all m -bit addresses.

A binary code closed under XOR is *linear*. Its parameters $[n, k, d_{\min}]$ mean n stored bits, k message bits, and minimum distance $d_{\min}$. Our code is $[8, 4, 4]$.

Each codeword has 9 possible readouts with at most one error: itself and its 8 single-bit changes. Unique correction requires these sets to be disjoint.

$$\underbrace{16}_{\text{codewords}} \underbrace{(1 + 8)}_{\text{readouts per codeword}} = 144 \leq \underbrace{2^8}_{\text{all readouts}} = 256.$$

For M length- n codewords correcting one error, the same count gives $M(1 + n) \leq 2^n$, the one-error *Hamming bound*.

The bound limits the number of messages at a fixed length under a worst-case error guarantee.

Erasure recovery and channel capacity

An *erasure* is a missing value with a known position, shown by ?. The same code gives two different examples:

readout	explanation
?1101001	only parity's codeword fits the seven known values
0??0?00?	both 00000000 and 01101001 fit

Now erase each position independently with probability ε . The *rate* is message bits per stored bit, $R = k/n$, here $4/8 = 1/2$. *Capacity* is the supremum of rates allowing error probability to approach zero as length grows.

Erasure recovery and channel capacity

An *erasure* is a missing value with a known position, shown by ?. The same code gives two different examples:

readout	explanation
?1101001	only parity's codeword fits the seven known values
0??0?00?	both 00000000 and 01101001 fit

Now erase each position independently with probability ε . The *rate* is message bits per stored bit, $R = k/n$, here $4/8 = 1/2$. *Capacity* is the supremum of rates allowing error probability to approach zero as length grows. For this binary erasure channel, $C = 1 - \varepsilon$. For example, at $\varepsilon = 1/4$, rates below $3/4$ can be made reliable.

Optimal erasure decoding finds the unique compatible codeword, or reports ambiguity. For every target $R \in (0, 1)$, a sequence of Reed–Muller codes has lengths growing, rates tending to R , and whole-word recovery probability tending to one when $\varepsilon < 1 - R$.

Kudekar et al., Reed–Muller Codes Achieve Capacity on Erasure Channels, Theorem 27.

Predicting a Boolean output with parity

How often does parity predict this table? Choose each three-bit input with probability $1/8$ (uniform sampling).

u	000	001	010	011	100	101	110	111
$r(u)$	1	1	1	0	1	0	0	1
$p(u)$	0	1	1	0	1	0	0	1

Seven agreements, one disagreement: prediction probability $7/8$.

Predicting a Boolean output with parity

How often does parity predict this table? Choose each three-bit input with probability $1/8$ (uniform sampling).

u	000	001	010	011	100	101	110	111
$r(u)$	1	1	1	0	1	0	0	1
$p(u)$	0	1	1	0	1	0	0	1

Seven agreements, one disagreement: prediction probability $7/8$. Encode a bit z by $(-1)^z$: $0 \mapsto +1$, $1 \mapsto -1$. The average sign product is a normalized *Fourier coefficient* of $(-1)^r$, at parity p :

$$\frac{1}{8} \sum_{u \in \mathbb{F}_2^3} (-1)^{r(u)} (-1)^{p(u)} = \frac{7-1}{8} = \frac{3}{4}, \quad \Pr[r = p] = \frac{1 + 3/4}{2} = \frac{7}{8}.$$

A cryptographic Boolean component computes one output bit from input bits. Accurate affine prediction is undesirable. This degree-three table still agrees with parity on $7/8$ of inputs.

Explicit construction. Forty-eight physical qubits encode three logical qubits and correct any one-qubit error.

Physical implementation. Twenty-six T and twenty-two $T^\dagger$ rotations implement logical CCZ without an additional logical correction.

Exact verification. Signed weights must give one phase throughout each encoded superposition. The row-intersection conditions verify this requirement.

In the worked example, affine-function weights proved classical distance. In the quantum construction, Boolean evaluation words and signed intersections determine both the encoding and the logical phase.

Backup: affine equivalence changes input coordinates

For m -bit inputs, let A be an invertible binary $m \times m$ matrix and $t \in \mathbb{F}_2^m$ a translation. Matrix operations here use XOR addition. Functions f, g are *input-affine equivalent* if

$$g(u) = f(Au \oplus t).$$

The bijection of input addresses permutes table positions. It preserves weight and, for nonzero functions, degree. Applying the same permutation to two tables preserves their distance. For a direction $v \in \mathbb{F}_2^m$, the derivatives satisfy

$$D_v g(u) = D_{Av} f(Au \oplus t).$$

Permuting physical sites preserves a code and its gate action when every generator word and rotation sign is transported. Adding a fixed vector to recorded check outcomes changes the check data instead.

Backup: the eight-qubit quantum distance is two

The toy code's checks detect every error acting on one physical qubit. A phase flip at site u is denoted Z_u . It changes the outcome of the all-site bit-flip check. Two such changes can cancel.

The *check syndrome* is the list of check outcomes changed by an error.

Put phase errors at addresses 000 and 100. The pair has zero check syndrome, but its action on every encoded basis state is

$$Z_{000}Z_{100}|a\rangle_L = (-1)^{a_1}|a\rangle_L.$$

Both physical words within a fixed complementary pair receive the same phase. The phase depends on the logical label a , so a superposition of different logical basis states can acquire a relative phase.

This undetected two-site phase error acts logically. The toy code therefore has quantum distance two.

Backup: the 48-qubit logical Boolean functions

On each block w , the function $\kappa_i(u, w)$ evaluates to the logical row K_i . All u_i are bits.

w	row	Boolean function
10	K_1	$u_1 \oplus u_3$
	K_2	$1 \oplus u_1 \oplus u_2 \oplus u_3 \oplus u_4$
	K_3	$1 \oplus u_1 \oplus u_1 u_2 \oplus u_1 u_4 \oplus u_3 u_4$
01	K_1	$1 \oplus u_3$
	K_2	$1 \oplus u_1 \oplus u_3 \oplus u_1 u_3 \oplus u_1 u_4$
	K_3	$u_1 \oplus u_1 u_2 \oplus u_2 u_3 \oplus u_2 u_4 \oplus u_3 u_4$
11	K_1	u_4
	K_2	$u_3 \oplus u_4 \oplus u_1 u_3 \oplus u_1 u_4$
	K_3	$u_1 \oplus u_3 \oplus u_1 u_4 \oplus u_2 u_3 \oplus u_2 u_4$

The current paper's fixed label map. Together with the six coordinate rows, these rows have rank nine.

Backup: the 48-qubit rotation signs

For $u \in \mathbb{F}_2^4$ and $w \in \mathbb{F}_2^2 \setminus \{0\}$, define

$$Q_1(u) = u_1 \oplus u_2 \oplus u_3 \oplus u_1u_3 \oplus u_2u_3 \oplus u_2u_4,$$

$$Q_2(u) = u_3 \oplus u_4 \oplus u_1u_2 \oplus u_1u_4 \oplus u_3u_4.$$

The sign function is

$$\gamma(u, w) = u_3 \oplus w_1Q_1(u) \oplus w_2Q_2(u).$$

Its weights on blocks 10, 01, 11 are 6, 6, 10. Value one selects $T^\dagger$, giving 22 inverse rotations and 26 forward rotations.

The sign polynomial specifies the physical rotations. The fixed logical rows and signed-weight calculation establish the logical gate.

Backup: the scope of the length exclusion

The paper studies *CSS codes*, whose checks separate into bit-flip and phase-flip types, with three logical qubits. The condition $d_Z \geq 3$ means that every undetected phase-type logical error uses at least three physical qubits.

The nine *Campbell–Howard conditions* specify even or odd parities for logical rows, stabilizer rows, and their pair and triple intersections. They are the overlap conditions for the paper’s quasi-transversal CCZ setting. Within this class, all lengths $n \leq 38$ are excluded, at every stabilizer rank.

physical length	conclusion
$n \leq 38$	no code in this specified class
$39 \leq n \leq 46$	existence remains open
$n = 47$	the particular cited code has no $T/T^\dagger$ -only CCZ pattern
$n = 48$	the constructed code has such a pattern

Scope: The construction does not prove global minimality. The 47-qubit statement concerns one specified code.

Current paper: length exclusion, fixed-label 47-qubit test, and explicit 48-qubit construction.

Quantum results: the author's current paper, *A 48-qubit native CCZ code from projective geometry and a length lower bound*. The talk uses its fixed logical functions, rotation signs, signed-overlap criterion, and qualified exclusion.

Memory context: IBM, Troubleshooting Memory issues. The eight-bit encoder is a teaching example.

Erasure capacity: Kudekar, Kumar, Mondelli, Pfister, Şaşıoğlu, and Urbanke, Reed–Muller Codes Achieve Capacity on Erasure Channels, Theorem 27.

Boolean fundamentals: Carlet, *Boolean Functions for Cryptography and Coding Theory*. Fourier agreement: O'Donnell, *Analysis of Boolean Functions*, Chapter 1.

Classical coding: MacWilliams–Sloane, *The Theory of Error-Correcting Codes*, and Huffman–Pless, *Fundamentals of Error-Correcting Codes*.